Week 4 Practice Problems

Problem 1 : Expected number of trials

As discussed in lecture, for repeated independent trials where each trial succeeds with probability p, the expected number of trials up to and including the first success is $1/p$.

(a) You roll a fair six-sided die repeatedly until you roll either a 5 or a 6. What is the expected number of rolls?

(b) You draw a card from a standard 52 card deck, with replacement (the card is put back in the deck each time, and reshuffled), repeatedly until you draw a heart. What is the expected number of draws?

(c) You flip a biased coin, with $P(\text{heads}) = 0.2$, repeatedly until it you get heads. What is the expected number of flips?

Solution. (a) Each value has probability of success $\frac{1}{6}$. Success (rolling a 5 or 6) has probability $p = \frac{1}{6} + \frac{1}{6} = \frac{2}{6} = \frac{1}{3}$. Expected number of rolls is $1/p = 3$.
(b) There are 13 hearts out of 52 cards, so $p = 13/52 = 1/4$. Expected number of draws is $1/p = 4$.
(c) Here, we are given that $p = 0.2$. Expected number of flips is $1/p = 1/0.2 = 5$.

Problem 2 : Priority queue via a sorted array

In lecture, you analyzed a priority queue that was implemented with an **unsorted doubly-linked list**. For that setup, you had that (excluding the runtime of mapping operations):

1. `Insert` is $\Theta(1)$
2. `Maximum` is $\Theta(n)$
3. `Extract-Max` is $\Theta(n)$
4. `Increase-Key` is $\Theta(1)$

Now consider implementing a priority queue with a **sorted array** instead. As with a heap, we can assume that the array is not necessarily always full. The elements of the priority queue are stored in increasing sorted order in positions 1 through `A.pqsize`, so the maximum element is always at position `A.pqsize`, and `A.pqsize` must be kept updated.

(a) `Insert(S,x,k)`: what is the worst case runtime?

(b) `Maximum(S)`: what is the worst case runtime?

(c) `Extract-Max(S)`: what is the worst case runtime?

(d) `Increase-Key(S,x,k)`: what is the worst case runtime, including the time to locate x via a linear search? What is the worst case runtime if, as above, we exclude the time for mapping operations (i.e. assume x's position is already known)?

Solution. (a) Finding the correct sorted position can be done in $\Theta(\log n)$ via binary search, but inserting there may require shifting up to all n existing elements over by one (e.g., inserting a new minimum shifts everything), and incrementing `A.pqsize`. So `Insert` is $\Theta(\log n) + \Theta(n) = \Theta(n)$ worst case.

(b) The maximum is always at position `A.pqsize`, so `Maximum` is $\Theta(1)$.

(c) Removing the element at position `A.pqsize` and decrementing `A.pqsize` requires no shifting at all, so `Extract-Max` is $\Theta(1)$.

(d) Including the search: locating x takes $\Theta(n)$ (linear search). Since x's key increases, its new sorted position moves toward the "large" end, requiring the elements between its old and new position to shift. This gives upto $\Theta(n)$ in the worst case. Total: $\Theta(n) + \Theta(n) = \Theta(n)$.

Excluding the search: even if x's position is already known for free, increasing its key still requires shifting elements to get back to sorted order. `Increase-Key` is therefore still $\Theta(n)$ in the worst case, even excluding mapping operations. This is different from the doubly-linked list case. There, once the search for x is excluded, nothing else needs to happen to the rest of the structure (it was never sorted to begin with), so `Increase-Key` drops all the way to $\Theta(1)$. For the sorted array, excluding the search removes only part of the cost, since the array still needs to stay sorted.

Operation	Doubly-linked unsorted list	Sorted array
Insert	$\Theta(1)$	$\Theta(n)$
Maximum	$\Theta(n)$	$\Theta(1)$
Extract-Max	$\Theta(n)$	$\Theta(1)$
Increase-Key	$\Theta(1)$	$\Theta(n)$

*(Table excludes the runtime of mapping operations. If mapping operations are **not** excluded, then `Increase-Key` takes $\Theta(n)$ for **both** the doubly-linked list and the sorted array.)*

Problem 3 : Revisiting the Heapsort lower bound

In lecuture, you argued that Heapsort's worst case running time is $\Omega(n \log n)$.

(a) Redo this argument using the first $n/4$ calls to `Max-Heapify` instead of the first $n/2$. How many elements does the heap have during each of these calls (at least), and what's the resulting lower bound on the height of that heap?

(b) Using your answer to (a), show that the first $n/4$ calls still give a total lower bound of $\Omega(n \log n)$.

(c) Why do you think the lecture's argument uses $n/2$ specifically, rather than $n/4$ (or some other fraction)?

Solution. (a) After i iterations of Heapsort's main loop, the heap has shrunk from n down to $n - i$ elements. During the first $n/4$ calls (i.e., $i = 0, \ldots, n/4 - 1$), the heap size is always $\geq n - n/4 = 3n/4$. A heap with $\geq 3n/4$ elements has height $\geq \log_2(3n/4) = \log_2 n + \log_2(3/4) \approx \log_2 n - 0.415$.

(b) Each of these first $n/4$ calls takes time $\geq c(\log_2 n - 0.415)$ in the worst case, so the total is at least

$$\frac{n}{4} \cdot c\,(\log_2 n - 0.415) = \Omega(n \log n),$$

since the leading term, $\frac{c}{4} n \log_2 n$, dominates the remaining $0.415 \cdot \frac{c}{4} n$ term. Thus, using the first $n/4$ calls still gives an $\Omega(n \log n)$ lower bound.

(c) Both fractions work, so $n/2$ isn't necessary for the argument to go through. All it does is give the cleanest arithmetic ($n/2$ elements remaining directly gives height $\geq \log_2 n - 1$, with no extra $\log_2(3/4)$ term to carry around).

Problem 4 : Classifying algorithms as in-place

Recall from lecture that a sorting algorithm is **in-place** if at most a constant number of elements of the input array are stored outside the input array at any time. For each algorithm below, say whether it is in-place, and briefly justify your answer. For all algorithms below, use the version discussed in lecture.

(a) Bubblesort.

(b) Mergesort, whose `Merge` step copies the two halves being merged into temporary arrays before merging them back.

(c) Heapsort.

(d) Quicksort.

Solution. (a) In-place. Bubblesort only ever swaps two elements of the array at a time, using at most a constant number of temporary variables. No extra array proportional to n is ever needed.

(b) Not in-place. Each call to `Merge` copies the subarrays being merged into extra arrays whose combined size is proportional to the number of elements being merged at that level. This is $\Theta(n)$ extra space at the top level of the recursion alone.

(c) In-place. Heapsort treats the input array itself as the heap, and both `Build-Max-Heap` and `Max-Heapify` only ever swap elements within that same array.

(d) In-place. The partitioning process only tracks a constant number of index variables (i, j, and the pivot value) and swaps elements within the array itself.